

Full Stack Web Development Lab

Week 10 - Record

Aim

To develop a Student Registration application using the MERN stack (MongoDB, Express.js, React.js and Node.js) that performs CRUD (Create, Read, Update, Delete) operations on student data stored in a MongoDB database.

Description

The application has two parts: a backend (Node.js + Express.js + Mongoose) and a frontend (React with Vite). The backend connects to the MongoDB database studentdb at mongodb://127.0.0.1:27017 and defines a Student schema with name, email, password, gender, country and languages (array). It exposes four REST routes on port 5000: GET /students (Read all students), POST /students (Create a student from req.body), PUT /students/:id (Update a student using findByIdAndUpdate) and DELETE /students/:id (Delete a student using findByIdAndDelete). The cors middleware allows the React app to call the server. The frontend shows a Student Registration form with text, email and password inputs, gender radio buttons, a country drop-down and language checkboxes; the values are held in state with useState and sent with fetch(). useEffect loads the students when the page opens and the Registered Students table shows every record with Edit and Delete buttons. Clicking Edit fills the form with that student's data and changes the Register button to Update (editId state), Delete removes the record, and Reset clears the form. If the server is not running, the message "Start the Express server first" is displayed. Flow: React Form → fetch() → Express Route → Mongoose Model → MongoDB → Response → Table Refresh.

Program

backend/package.json

```
{
  "name": "mern-registration-backend",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "express": "^5.1.0",
    "mongoose": "^8.18.0"
  }
}
```

backend/server.js (Express + MongoDB server)

```
import express from "express";
import mongoose from "mongoose";
import cors from "cors";

const app = express();

app.use(cors());
app.use(express.json());

// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/studentdb")
  .then(() => {
    console.log("MongoDB connected");
  })
  .catch((error) => {
    console.log("MongoDB connection error:", error.message);
  });

// Schema means structure of Data to be stored in database
const studentSchema = new mongoose.Schema({
  name: String,
  email: String,
```

```
password: String,
gender: String,
country: String,
languages: [String]
});

// Model
const Student = mongoose.model("Student", studentSchema);

// Test route
app.get("/", (req, res) => {
  res.send("Express server is working");
});

// READ
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// CREATE
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();

  res.json({
    message: "Registration successful"
  });
});

// UPDATE
app.put("/students/:id", async (req, res) => {
  await Student.findByIdAndUpdate(
    req.params.id,
    req.body
  );

  res.json({
    message: "Student updated successfully"
  });
});

// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);

  res.json({
    message: "Student deleted successfully"
  });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});
```

frontend/package.json

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.18",
    "@types/react-dom": "^19.2.7",
    "@vitejs/plugin-react": "^6.1.1",
    "eslint": "^10.10.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.6",
    "globals": "^17.12.0",
    "vite": "^8.3.0"
  }
}
```

frontend/vite.config.js

```
import react from '@vitejs/plugin-react'
import { defineConfig } from 'vite'

// https://vite.dev/config/
export default defineConfig({
  plugins: [react()],
})
```

frontend/index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>frontend</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

frontend/src/main.jsx (React entry point)

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

frontend/src/App.jsx (Registration form + CRUD)

```
import heroImg from './assets/hero.png'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import './App.css'

import { useEffect, useState } from "react";

const API = "http://localhost:5000";

function App() {

  const emptyForm = {
    name: "",
    email: "",
    password: "",
    gender: "",
    country: "",
    languages: []
  };

  const [form, setForm] = useState(emptyForm);

  const [students, setStudents] = useState([]);

  const [message, setMessage] = useState("");

  const [editId, setEditId] = useState("");

  // Get students when page opens
  useEffect(() => {
    loadStudents();
  }, []);

  // READ
  function loadStudents() {

    fetch(API + "/students")
      .then(response => response.json())
      .then(data => {
        setStudents(data);
      })
      .catch(error => {
        console.log(error);
        setMessage("Start the Express server first");
      });
  }
}
```

```
    });  
  }  
  
  // Text, email, password, radio and select  
  function handleChange(event) {  
  
    const { name, value } = event.target;  
  
    setForm({  
      ...form,  
      [name]: value  
    });  
  }  
}  
  
// Checkbox  
function handleLanguage(event) {  
  
  const value = event.target.value;  
  const checked = event.target.checked;  
  
  let languages = form.languages;  
  
  if (checked) {  
    languages = [...languages, value];  
  } else {  
    languages = languages.filter(  
      item => item !== value  
    );  
  }  
  
  setForm({  
    ...form,  
    languages: languages  
  });  
}  
  
// CREATE or UPDATE  
function saveStudent(event) {  
  
  event.preventDefault();  
  
  let url = API + "/students";  
  
  let method = "POST";  
  
  if (editId !== "") {  
  
    url = API + "/students/" + editId;  
  
    method = "PUT";  
  }  
  
  fetch(url, {  
    method: method,  
  
    headers: {  
      "Content-Type": "application/json"  
    },  
  
    body: JSON.stringify(form)  
  })  
  .then(response => response.json())  
  .then(data => {  
  
    setMessage(data.message);  
  
    setForm(emptyForm);  
  
    setEditId("");  
  
    loadStudents();  
  
  })  
  .catch(error => {  
  
    console.log(error);  
  
    setMessage("Could not connect to Express");  
  
  });  
}  
// EDIT  
function editStudent(student) {
```

```
setForm({
  name: student.name,
  email: student.email,
  password: student.password,
  gender: student.gender,
  country: student.country,
  languages: student.languages
});

setEditId(student._id);

setMessage("Edit the data and click Update");
}
```

```
// DELETE
function deleteStudent(id) {

  fetch(API + "/students/" + id, {
    method: "DELETE"
  })
  .then(response => response.json())
  .then(data => {

    setMessage(data.message);

    loadStudents();

  });
}
```

```
// RESET
function resetForm() {

  setForm(emptyForm);

  setEditId("");

  setMessage("");
}
```

```
return (

  <div className="page">

    <div className="form-box">

      <h1>Student Registration</h1>

      <form onSubmit={saveStudent}>

        <label>Name</label>

        <input
          name="name"
          value={form.name}
          onChange={handleChange}
          required
        />

        <label>Email</label>

        <input
          type="email"
          name="email"
          value={form.email}
          onChange={handleChange}
          required
        />

        <label>Password</label>

        <input
          type="password"
          name="password"
          value={form.password}
          onChange={handleChange}
          required
        />

        <label>Gender</label>
      <div className="row">
```

```
<label>
  <input
    type="radio"
    name="gender"
    value="Male"
    checked={form.gender === "Male"}
    onChange={handleChange}
  />
  Male
</label>

<label>
  <input
    type="radio"
    name="gender"
    value="Female"
    checked={form.gender === "Female"}
    onChange={handleChange}
  />
  Female
</label>
</div>

<label>Country</label>

<select
  name="country"
  value={form.country}
  onChange={handleChange}
  required
>

  <option value="">
    Select Country
  </option>

  <option value="India">
    India
  </option>

  <option value="USA">
    USA
  </option>

  <option value="UK">
    UK
  </option>

  <option value="Australia">
    Australia
  </option>
</select>

<label>Languages</label>

<div className="row">

  <label>
    <input
      type="checkbox"
      value="English"
      checked={form.languages.includes("English")}
      onChange={handleLanguage}
    />
    English
  </label>

  <label>
    <input
      type="checkbox"
      value="Hindi"
      checked={form.languages.includes("Hindi")}
      onChange={handleLanguage}
    />
    Hindi
  </label>

  <label>
    <input
```

```

        type="checkbox"
        value="Telugu"
        checked={form.languages.includes("Telugu")}
        onChange={handleLanguage}
      />
      Telugu
    </label>

  </div>

  <button type="submit">
    {editId === "" ? "Register" : "Update"}
  </button>

  <button
    type="button"
    onClick={resetForm}
  >
    Reset
  </button>

</form>

{message && (
  <div className="message">
    {message}
  </div>
)}
)}

</div>

<div className="data-box">

  <h2>Registered Students</h2>

  <table>

    <thead>

      <tr>
        <th>Name</th>
        <th>Email</th>
        <th>Gender</th>
        <th>Country</th>
        <th>Languages</th>
        <th>Actions</th>
      </tr>

    </thead>

    <tbody>

      {students.map(student => (
        <tr key={student._id}>
          <td>{student.name}</td>
          <td>{student.email}</td>
          <td>{student.gender}</td>
          <td>{student.country}</td>
          <td>
            {student.languages.join(", ")}
          </td>
          <td>
            <button
              onClick={() => editStudent(student)}
            >
              Edit
            </button>

            <button
              onClick={() =>
                deleteStudent(student._id)

```

```

        }
        >
        Delete
    </button>

</td>

</tr>

)}}
</tbody>

</table>

</div>

</div>
);
}

```

```
export default App;
```

frontend/src/App.css

```

.page {
  display: flex;
  gap: 24px;
  max-width: 1100px;
  margin: 20px auto;
}

.form-box,
.data-box {
  padding: 20px;
  background: white;
  border: 1px solid #ddd;
}

.form-box {
  width: 300px;
  flex-shrink: 0;
}

.form-box form {
  display: flex;
  flex-direction: column;
  gap: 8px;
}

.form-box input:not([type='radio']):not([type='checkbox']),
.form-box select {
  padding: 8px;
}

.row {
  display: flex;
  gap: 12px;
}

button {
  padding: 8px;
  cursor: pointer;
}

.data-box {
  flex: 1;
  overflow-x: auto;
}

table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
}

th,
td {
  padding: 8px;
  border-bottom: 1px solid #ddd;
}

.message {
  margin-top: 12px;
}

@media (max-width: 700px) {

```



```

body {
  padding: 10px;
}

.page {
  flex-direction: column;
}

.form-box {
  width: auto;
}

```

frontend/src/index.css

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  padding: 20px;
  font-family: Arial, sans-serif;
  background: #f4f6f5;
  color: #222;
}

button,
input,
select {
  font: inherit;
}

```

Details about the Program

Where Saved	All files are saved inside the project folder FL_LAB_ANITS/fs_week_10/ in the GitHub repository raviteja-codehub/FL_LAB_ANITS — the backend/ folder (server.js, package.json) and the frontend/ folder (index.html, vite.config.js, package.json and src/ with main.jsx, App.jsx, App.css, index.css). node_modules is not uploaded.
Where Executed	The Express server runs on Node.js at http://localhost:5000 and stores data in the local MongoDB database studentdb. The React frontend runs on the Vite development server and is opened in a web browser (Google Chrome / Edge / Firefox) at http://localhost:5173 . MongoDB must be running on the computer.
How Executed	1. Start MongoDB on the local computer (port 27017). 2. Open a terminal in fs_week_10/backend, run npm install and then node server.js; the console prints "MongoDB connected" and "Server running on port 5000". 3. Open another terminal in fs_week_10/frontend, run npm install and then npm run dev. 4. Open http://localhost:5173, fill the form and click Register; the student appears in the Registered Students table. Use Edit, Update and Delete to modify or remove records.

Output

After starting MongoDB, the Express server and the React app, and opening <http://localhost:5173>, the registration form and the registered students list appear as shown below:

Student Registration Page (form filled)

Student Registration

Name
Raviteja Pasupuleti

Email
ravi@gmail.com

Password

Gender
☒ Male ☐ Female

Country
India

Languages
☐ English ☐ Hindi ☒ Telugu

Register

Reset

Start the Express server first

Registered Students

Name	Email	Gender	Country	Languages	Actions
------	-------	--------	---------	-----------	---------

Registered Students (after registering students)

Student Registration

Name

Email

Password

Gender
☐ Male ☐ Female

Country
Select Country

Languages
☐ English ☐ Hindi ☐ Telugu

Register

Reset

Registration successful

Registered Students

Name	Email	Gender	Country	Languages	Actions
Raviteja Pasupuleti	ravi@gmail.com	Male	India	Telugu	<button>Edit</button> <button>Delete</button>
Sravani K	sravani@gmail.com	Female	USA	English, Hindi	<button>Edit</button> <button>Delete</button>
Kiran Kumar	kiran@gmail.com	Male	UK	English, Telugu	<button>Edit</button> <button>Delete</button>

Edit (form filled with the student's data and Update button)

Student Registration

Name

Email

Password

Gender

☐ Male ☒ Female

Country

Languages

☒ English ☒ Hindi ☐ Telugu

Update

Reset

Edit the data and click Update

Registered Students

Name	Email	Gender	Country	Languages	Actions
Raviteja Pasupuleti	ravi@gmail.com	Male	India	Telugu	Edit Delete
Sravani K	sravani@gmail.com	Female	USA	English, Hindi	Edit Delete
Kiran Kumar	kiran@gmail.com	Male	UK	English, Telugu	Edit Delete

After Update and Delete

Student Registration

Name

Email

Password

Gender

☐ Male ☐ Female

Country

Languages

☐ English ☐ Hindi ☐ Telugu

Register

Reset

Student deleted successfully

Registered Students

Name	Email	Gender	Country	Languages	Actions
Raviteja Pasupuleti	ravi@gmail.com	Male	India	Telugu	Edit Delete
Sravani K	sravani@gmail.com	Female	USA	English, Hindi	Edit Delete